

Part 4:

1. I would join the trips table on the dropoff area to find drivers that have both visited the same area. I would then filter out the original driver and then group by the second driver, then count the number of areas that they share to get the result.
2. The relationships between drivers and areas are stored directly in Neo4j so we can follow connections from one driver to areas and then also to other drivers. This will avoid multiple joins and makes the query better to write. Since we will have an index free adjacency each node directly connects to its neighbors which makes the multistep traversals more efficient and faster.
3. Neo4j underperforms when the queries do not traverse relationships and require scanning large portions of the graph or performing large aggregations over many nodes and relationships. For these cases, this graph structure does not provide much benefit and the system spends a lot of time processing data rather than following the connections. This is what reduces the advantage of graph traversals which leads it to underperform.